

Percona Operator for PostgreSQL on Kubernetes

Environment Analysis, Architecture, Production CRDs & Troubleshooting

Clusters: pgkubcluster (374d) & salt-control-postgres (90d)

Operator v2.8.x | PostgreSQL 14-18 | Patroni + pgBouncer + pgBackRest + PMM

Table of Contents

1. Your Environment Analysis & Health Check
2. Architecture Overview (With Your Clusters)
3. How the Operator Works
4. High Availability & Automatic Failover
5. Backup & Restore with pgBackRest
6. Production CRDs: Vacuum, Analyze & Maintenance
7. On-Demand Backup & Restore CRDs
8. Database Monitoring: Locks, Deadlocks & Bloat
9. Troubleshooting Guide (Environment-Specific)
10. Quick Reference & Commands

1 Your Environment Analysis & Health Check

Based on the **kubecttl get -A all** output from your two Kubernetes clusters, here is a comprehensive analysis of your Percona Operator for PostgreSQL environment.

Cluster 1: pgkubcluster

Property	Value	Status
Cluster Name	pgkubcluster	Active
Namespace	postgres-operator	OK
Age	374 days	Production mature
Nodes	6 (3 CP + 3 Worker)	Based on DaemonSets
Operator	percona-postgresql-operator-568594ddb9-c6gr2	1/1 Running
Instance 1 (Primary)	pgkubcluster-i1-rvcz-0	5/5 Running
Instance 2 (Replica)	pgkubcluster-i2-d6hm-0	4/5 NOT READY
pgBouncer	2 replicas (b7f656889-k4j7c, s62tn)	2/2 Running
Repo Host	pgkubcluster-repo-host-0	2/2 Running
pgBouncer Service	NodePort 10.105.44.229:5432:30543	Exposed
Primary Service	ClusterIP None:5432	Internal
Replicas Service	ClusterIP 10.109.229.130:5432	Internal
Backup CronJob	pgbackupmonitor (50 23 * * *)	Daily 23:50
PMM Server	pmm-0 in percona-pmm namespace	1/1 Running
Monitoring	Full Prometheus stack + Grafana	Running

HEALTH ALERT - Cluster 1

CRITICAL: pgkubcluster-i2-d6hm-0 shows 4/5 containers ready!

- StatefulSet pgkubcluster-i2-d6hm shows READY: 0/1
- One container (likely PMM sidecar or pgBackRest) is failing
- 3 pgbackupmonitor jobs FAILED (19d, 18d, 17d ago)
- Recent pgbackupmonitor jobs (2d17h, 41h, 17h) are Completing OK

ACTION REQUIRED: Run 'kubecttl describe pod pgkubcluster-i2-d6hm-0 -n postgres-operator' to identify which container is not ready and check its logs.

Cluster 2: salt-control-postgres

Property	Value	Status
Cluster Name	salt-control-postgres	Active
Namespace	postgres-operator	OK
Age	90 days	Production
Nodes	10 (5 CP + 5 Worker)	Based on DaemonSets
Operator	percona-postgresql-operator-76db54796d-zlsqc	1/1 Running
Instance 1	salt-control-postgres-i1-hpkf-0	5/5 Running
Instance 2	salt-control-postgres-i2-5tf6-0	5/5 Running
Instance 3	salt-control-postgres-i3-ptk4-0	5/5 Running
pgBouncer Deployment	salt-control-postgres-pgbouncer	0/0 SCALED DOWN
HA Service	NodePort 10.106.107.207:5432:30543	Exposed
pgBouncer Service	NodePort 10.107.93.161:5432:32005	Exposed
Repo Host	salt-control-postgres-repo-host-0	2/2 Running
Backup CronJob	pgbackupmonitor (50 23 * * *)	Daily 23:50
Backups	13 completed, duration ~5m each	All Complete
PMM Server	pmm-0 in percona-pmm namespace	1/1 Running

WARNING - Cluster 2: pgBouncer Down

WARNING: pgBouncer deployment is scaled to 0/0 replicas!

- deployment.apps/salt-control-postgres-pgbouncer shows READY: 0/0
- replicaset salt-control-postgres-pgbouncer-96ccb8c9 DESIRED: 0
- Clients may be connecting directly via HA service (NodePort:30543)
- Direct connections bypass connection pooling benefits

RECOMMENDATION: Scale pgBouncer back up unless intentionally disabled.

Check CR: spec.proxy.pgBouncer.replicas should be >= 2 for production.

Shared Infrastructure (Both Clusters)

Shared Platform Components

CNI: Cilium (DaemonSet across all nodes)

CSI: ssccloud (controller + node DaemonSets for persistent storage)

DNS: CoreDNS (2 replicas)

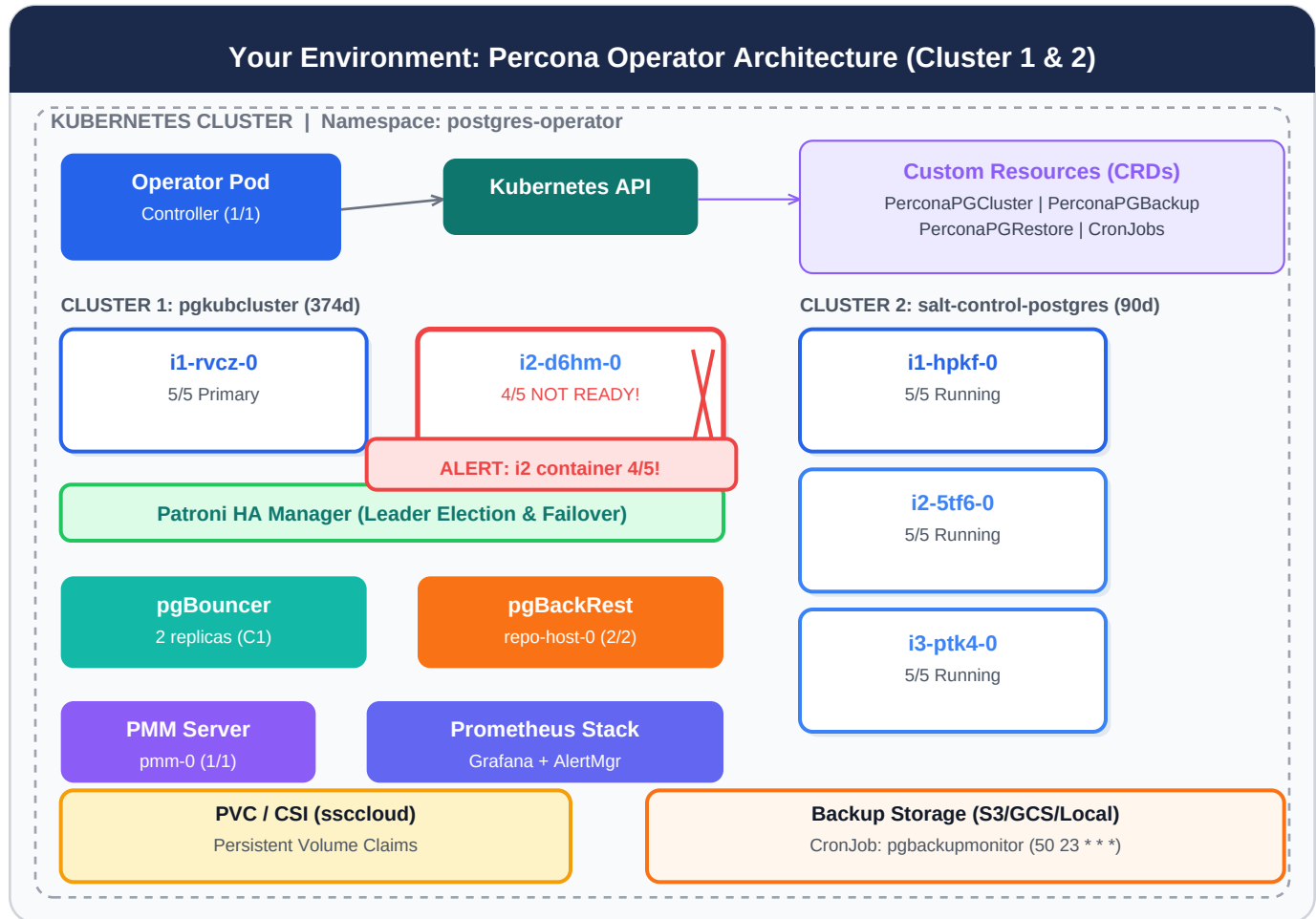
Monitoring: kube-prometheus-stack (Prometheus, Grafana, AlertManager, node-exporter)

PMM: Percona Monitoring & Management (pmm-0 StatefulSet in percona-pmm namespace)

Cloud: cloud-system auth-proxy pods (likely cloud provider managed K8s)

2 Architecture Overview (With Your Clusters)

The Percona Operator for PostgreSQL extends Kubernetes with Custom Resource Definitions (CRDs) to automate the full lifecycle of PostgreSQL clusters. Below is the architecture diagram reflecting your actual environment, showing both clusters with their instances, services, and supporting components.



Components Per Pod (5 Containers)

Each PostgreSQL instance pod runs 5 containers, which explains the 5/5 READY status you see in your clusters:

#	Container	Purpose
1	database	Main PostgreSQL server process
2	replication-cert-copy	Manages TLS certificates for replication
3	pgbackrest	pgBackRest agent for backup operations
4	pgbackrest-config	Configuration sidecar for pgBackRest
5	pmm-client	PMM monitoring agent (metrics collection)

Why 4/5 on i2-d6hm-0?

Your Cluster 1 instance i2-d6hm-0 showing 4/5 means ONE of these containers is failing.

Most commonly it's the pmm-client or pgbackrest container. Check with:

```
kubectl describe pod pgkubcluster-i2-d6hm-0 -n postgres-operator
```

```
kubectl logs pgkubcluster-i2-d6hm-0 -c pmm-client -n postgres-operator
```

3 How the Operator Works

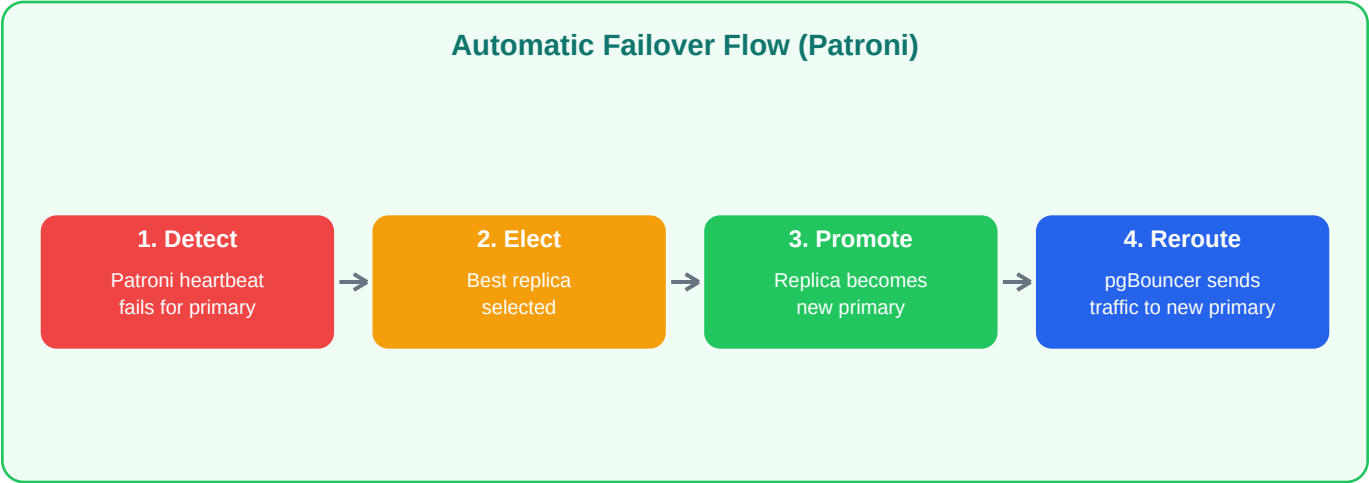
The Percona Operator implements the Kubernetes Operator pattern: it watches for changes to Custom Resources (CRs) via the Kubernetes API and continuously reconciles the actual state with the desired state. When you apply a PerconaPGCluster CR, the Operator creates StatefulSets, Services, Secrets, ConfigMaps, CronJobs, and other resources needed for a production-grade PostgreSQL cluster.

Resources Created by the Operator (Visible in Your Clusters)

Resource Type	Your Cluster 1 Examples	Purpose
StatefulSet	pgkubcluster-i1-rvcz, i2-d6hm, repo-host	DB instances + backup repo
Deployment	percona-postgresql-operator, pgkubcluster-pgbouncer	Operator + connection pooler
Service (ClusterIP)	pgkubcluster-primary, -replicas, -pods, -ha-config	Internal routing
Service (NodePort)	pgkubcluster-pgbouncer (5432:30543)	External access
CronJob	pgkubcluster-pgbackupmonitor (50 23 * * *)	Daily backup monitor
Job	pgkubcluster-backup-* (multiple completed)	On-demand backups
Secret	Auto-generated TLS certs + user credentials	Security
PVC	Per-instance persistent storage via ssccloud CSI	Data persistence

4 High Availability & Automatic Failover

Patroni manages PostgreSQL HA within your clusters. Each cluster has one primary (leader) handling read/write traffic and one or more replicas for read-only queries and failover candidates. Patroni uses Kubernetes endpoints for leader election (visible as the ha-config service). When the primary fails, Patroni automatically promotes the healthiest replica.

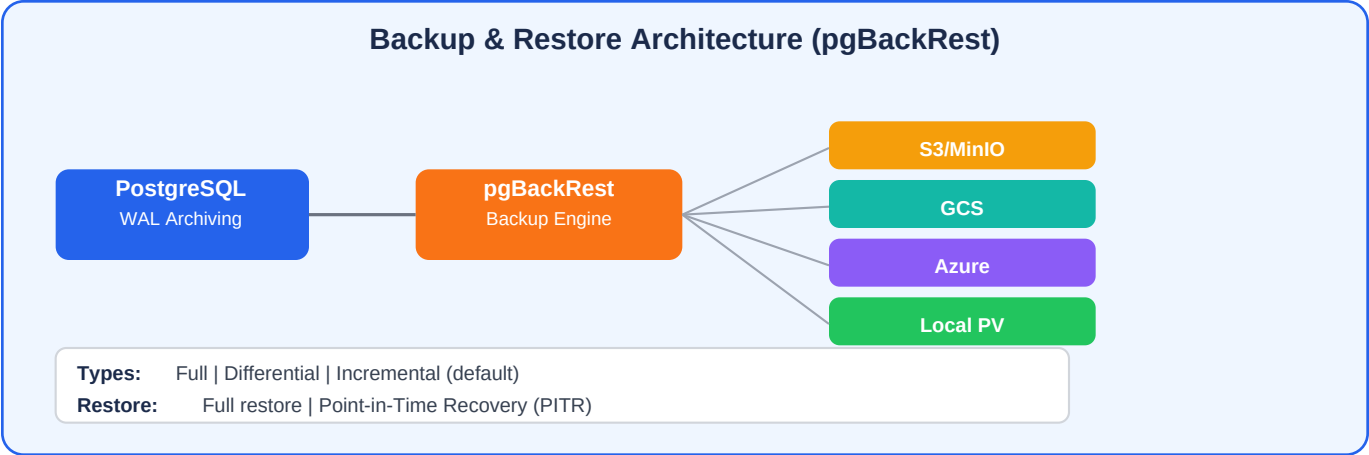


Your HA Configuration

Cluster	Instances	Primary	Replicas	HA Service
pgkubcluster	2 (i1 + i2)	i1-rvcz-0 (5/5)	i2-d6hm-0 (4/5!)	ha-config (ClusterIP)
salt-control-postgres	3 (i1 + i2 + i3)	i1-hpkf-0 (5/5)	i2-5tf6-0, i3-ptk4-0	salt-control-postgres-ha (NodePort)

Cluster 2 (salt-control-postgres) has stronger HA with 3 instances spread across more nodes. Cluster 1 should resolve the i2 container issue to maintain full HA capability.

5 Backup & Restore with pgBackRest



Your Backup Status

Cluster	Backup Jobs	Status	Avg Duration	Schedule
pgkubcluster	6 backup jobs	All Completed (7-9s)	~8 seconds	50 23 * * * (daily)
pgkubcluster	pgbackupmonitor (19d,18d,17d)	3 FAILED	N/A	See troubleshooting
pgkubcluster	pgbackupmonitor (recent)	3 Completed (4-6s)	~5 seconds	Recovered
salt-control-postgres	13 backup jobs	All Completed	~5m 20s	50 23 * * * (daily)
salt-control-postgres	pgbackupmonitor (3 jobs)	All Completed (3-6s)	~4 seconds	OK

Backup Health Assessment

Cluster 1 backups complete in ~8 seconds (likely incremental, small dataset).
Cluster 2 backups take ~5 minutes (larger dataset or full backups).
The 3 failed pgbackupmonitor jobs in Cluster 1 (19d-17d ago) suggest a transient issue that has since been resolved, as recent monitor jobs are completing successfully.

6

Production CRDs: Vacuum, Analyze & Maintenance

PostgreSQL requires regular maintenance operations like VACUUM and ANALYZE to reclaim storage, prevent transaction ID wraparound, and update query planner statistics. Below are production-ready CRDs and configurations for automating these tasks with the Percona Operator.

6.1 Patroni Dynamic Configuration for Autovacuum Tuning

The most effective way to manage VACUUM/ANALYZE is through PostgreSQL's built-in autovacuum, tuned via Patroni dynamic configuration in your PerconaPGCluster CR:

```
apiVersion: pgv2.percona.com/v2
kind: PerconaPGCluster
metadata:
  name: pgkubcluster
  namespace: postgres-operator
spec:
  patroni:
    dynamicConfiguration:
      postgresql:
        parameters:
          # ■■ Autovacuum Tuning (Production) ■■
          autovacuum: "on"
          autovacuum_max_workers: "5"
          autovacuum_naptime: "30s"
          autovacuum_vacuum_threshold: "50"
          autovacuum_vacuum_scale_factor: "0.05"
          autovacuum_analyze_threshold: "50"
          autovacuum_analyze_scale_factor: "0.05"
          autovacuum_vacuum_cost_delay: "2ms"
          autovacuum_vacuum_cost_limit: "1000"
          # ■■ Prevent Transaction ID Wraparound ■■
          autovacuum_freeze_max_age: "300000000"
          vacuum_freeze_min_age: "500000000"
          # ■■ Logging ■■
          log_autovacuum_min_duration: "1000"
```

Parameter	Value	Why
autovacuum_max_workers	5	More parallel vacuum workers for large DBs
autovacuum_naptime	30s	Check for work every 30s (default 1min)
autovacuum_vacuum_scale_factor	0.05	Vacuum after 5% rows changed (default 20%)
autovacuum_analyze_scale_factor	0.05	Analyze after 5% rows changed (default 10%)
autovacuum_vacuum_cost_delay	2ms	Reduce I/O throttling for faster vacuum
autovacuum_vacuum_cost_limit	1000	Allow more I/O per vacuum cycle
autovacuum_freeze_max_age	300M	Trigger anti-wraparound vacuum earlier

log_autovacuum_min_duration	1000	Log vacuum operations taking > 1 second
-----------------------------	------	---

6.2 Manual VACUUM / ANALYZE via CronJob

For targeted maintenance on specific tables, create a Kubernetes CronJob that connects through pgBouncer:

```
apiVersion: batch/v1
kind: CronJob
metadata:
  name: pg-maintenance-vacuum
  namespace: postgres-operator
spec:
  schedule: "0 3 * * 0" # Every Sunday at 3 AM
  concurrencyPolicy: Forbid
  jobTemplate:
    spec:
      template:
        spec:
          containers:
            - name: vacuum-job
              image: percona/percona-postgresql-operator:2.8.2-ppg16-postgres
              command: ["psql"]
              args:
                - "$(PGBOUNCER_URI)"
                - "-c"
                - |
                  VACUUM (VERBOSE, ANALYZE);
              envFrom:
                - secretRef:
                    name: pgkubcluster-pguser-pgkubcluster
          restartPolicy: OnFailure
```

7

On-Demand Backup & Restore CRDs

7.1 On-Demand Full Backup

```
apiVersion: pgv2.percona.com/v2
kind: PerconaPGBBackup
metadata:
  name: pgkubcluster-backup-ondemand-full
  namespace: postgres-operator
spec:
  pgCluster: pgkubcluster
  repoName: repo1
  options:
    - --type=full
    - --log-level-console=info
```

7.2 On-Demand Differential Backup

```
apiVersion: pgv2.percona.com/v2
kind: PerconaPGBBackup
metadata:
  name: pgkubcluster-backup-ondemand-diff
  namespace: postgres-operator
spec:
  pgCluster: pgkubcluster
  repoName: repo1
  options:
    - --type=diff
```

7.3 On-Demand Incremental Backup

```
apiVersion: pgv2.percona.com/v2
kind: PerconaPGBBackup
metadata:
  name: pgkubcluster-backup-ondemand-incr
  namespace: postgres-operator
spec:
  pgCluster: pgkubcluster
  repoName: repo1
  options:
    - --type=incr
```

7.4 Restore from Latest Backup

```
apiVersion: pgv2.percona.com/v2
kind: PerconaPGRestore
metadata:
  name: pgkubcluster-restore-latest
  namespace: postgres-operator
spec:
  pgCluster: pgkubcluster
  repoName: repo1
  options:
    - --type=immediate
```

7.5 Point-in-Time Recovery (PITR)

```
apiVersion: pgv2.percona.com/v2
kind: PerconaPGRestore
metadata:
  name: pgkubcluster-restore-pitr
  namespace: postgres-operator
spec:
  pgCluster: pgkubcluster
  repoName: repo1
  options:
    - --type=time
    - --target="2026-03-06 22:00:00+05:30"
    - --set=20260306-172350F
```

7.6 Backup Retention Policy in CR

```
spec:
  backups:
    pgbackrest:
      global:
        repo1-retention-full: "7"
        repo1-retention-full-type: count
        repo1-retention-diff: "4"
        repo1-retention-archive: "7"
        repo1-retention-archive-type: full
      repos:
        - name: repo1
      schedules:
        full: "0 1 * * 0" # Weekly full Sunday 1 AM
        differential: "0 1 * * 1-6" # Daily diff Mon-Sat
```

Action	Command
Apply backup CR	kubectl apply -f backup-full.yaml -n postgres-operator
Check backup status	kubectl get perconapgbackup -n postgres-operator
Apply restore CR	kubectl apply -f restore-pitr.yaml -n postgres-operator
Check restore status	kubectl get perconapgrestore -n postgres-operator
List pgBackRest info	kubectl exec pgkubcluster-i1-rvcz-0 -c database -- pgbackrest info
List backups in repo	kubectl exec pgkubcluster-repo-host-0 -c pgbackrest -- pgbackrest info

8 Database Monitoring: Locks, Deadlocks & Bloat

Below are production-tested SQL monitoring queries and Kubernetes commands for monitoring your Percona PostgreSQL clusters. These can be run via `kubectl exec` or through a monitoring CronJob.

8.1 Active Locks Monitoring

```
-- View all current locks with blocking information
SELECT
  pid,
  username,
  pg_blocking_pids(pid) AS blocked_by,
  query AS current_query,
  state,
  wait_event_type,
  wait_event,
  now() - query_start AS query_duration
FROM pg_stat_activity
WHERE state != 'idle'
  AND pid != pg_backend_pid()
ORDER BY query_duration DESC;
```

8.2 Lock Contention / Blocking Queries

```
-- Identify blocking and blocked queries
SELECT
  blocked.pid AS blocked_pid,
  blocked.username AS blocked_user,
  blocking.pid AS blocking_pid,
  blocking.username AS blocking_user,
  blocked.query AS blocked_query,
  blocking.query AS blocking_query,
  now() - blocked.query_start AS blocked_duration
FROM pg_stat_activity AS blocked
JOIN pg_stat_activity AS blocking
  ON blocking.pid = ANY(pg_blocking_pids(blocked.pid))
WHERE blocked.state = 'active';
```

8.3 Deadlock Detection & Configuration

```
-- Check deadlock settings
SHOW deadlock_timeout; -- Default: 1s
SHOW log_lock_waits;   -- Should be: on

-- Count deadlocks per database
SELECT datname, deadlocks, conflicts
FROM pg_stat_database
WHERE deadlocks > 0
ORDER BY deadlocks DESC;

-- Enable deadlock logging in CR patroni config:
-- patroni.dynamicConfiguration.postgresql.parameters:
--   deadlock_timeout: "1s"
--   log_lock_waits: "on"
--   lock_timeout: "30s"
```

8.4 Table Bloat Detection

```
-- Estimate table bloat (dead tuples)
SELECT
  schemaname || '.' || relname AS table_name,
  n_live_tup AS live_rows,
  n_dead_tup AS dead_rows,
  CASE WHEN n_live_tup > 0
    THEN round(100.0 * n_dead_tup / n_live_tup, 2)
    ELSE 0 END AS dead_pct,
  last_vacuum,
  last_autovacuum,
  last_analyze,
  last_autoanalyze
FROM pg_stat_user_tables
WHERE n_dead_tup > 1000
ORDER BY n_dead_tup DESC
LIMIT 20;
```


8.5 Index Bloat Detection

```
-- Estimate index bloat
SELECT
  schemaname || '.' || indexrelname AS index_name,
  schemaname || '.' || relname AS table_name,
  pg_size_pretty(pg_relation_size(indexrelid)) AS index_size,
  idx_scan AS index_scans,
  idx_tup_read,
  idx_tup_fetch
FROM pg_stat_user_indexes
WHERE idx_scan = 0 AND pg_relation_size(indexrelid) > 8192
ORDER BY pg_relation_size(indexrelid) DESC
LIMIT 20;
```

8.6 Long-Running Queries

```
-- Find queries running longer than 5 minutes
SELECT
  pid, username, datname, state,
  now() - query_start AS duration,
  left(query, 100) AS query_preview
FROM pg_stat_activity
WHERE state = 'active'
  AND now() - query_start > interval '5 minutes'
  AND pid != pg_backend_pid()
ORDER BY duration DESC;

-- Kill a long-running query (replace <pid>)
SELECT pg_cancel_backend(<pid>); -- Graceful cancel
SELECT pg_terminate_backend(<pid>); -- Force terminate
```

8.7 kubectl Commands to Run These Queries

Purpose	kubectl Command
Run SQL on Primary (C1)	kubectl exec pgkubcluster-i1-rvcz-0 -c database -n postgres-operator -- psql -c "<SQL_QUERY>"
Run SQL on Primary (C2)	kubectl exec salt-control-postgres-i1-hpkf-0 -c database -n postgres-operator -- psql -c "<SQL_QUERY>"
Interactive psql (C1)	kubectl exec -it pgkubcluster-i1-rvcz-0 -c database -n postgres-operator -- psql
Patroni status (C1)	kubectl exec pgkubcluster-i1-rvcz-0 -c database -n postgres-operator -- patronictl list
Check replication lag	kubectl exec <pod> -c database -- psql -c "SELECT * FROM pg_stat_replication;"

8.8 Monitoring CronJob for Automated Alerts

```
apiVersion: batch/v1
kind: CronJob
metadata:
```

```
name: pg-monitor-bloat-locks
namespace: postgres-operator
spec:
  schedule: "*/15 * * * *" # Every 15 minutes
  concurrencyPolicy: Forbid
  jobTemplate:
    spec:
      template:
        spec:
          containers:
            - name: monitor
              image: percona/percona-postgresql-operator:2.8.2-ppg16-postgres
              command: ["/bin/sh", "-c"]
              args:
                - |
                  echo "=== Lock Check ==="
                  psql "$PGBOUNCER_URI" -c "SELECT pid, username,
                    pg_blocking_pids(pid), state, query FROM
                    pg_stat_activity WHERE cardinality(
                    pg_blocking_pids(pid)) > 0;"
                  echo "=== Bloat Check ==="
                  psql "$PGBOUNCER_URI" -c "SELECT relname,
                    n_dead_tup, last_autovacuum FROM
                    pg_stat_user_tables WHERE n_dead_tup > 10000
                    ORDER BY n_dead_tup DESC LIMIT 10;"
                  echo "=== Deadlock Count ==="
                  psql "$PGBOUNCER_URI" -c "SELECT datname, deadlocks
                    FROM pg_stat_database WHERE deadlocks > 0;"
              envFrom:
                - secretRef:
                    name: pgkubcluster-pguser-pgkubcluster
              restartPolicy: OnFailure
```

9

Troubleshooting Guide (Environment-Specific)

Issue 1: pgkubcluster-i2-d6hm-0 Container 4/5 Not Ready

Symptoms

OBSERVED: Pod shows 4/5 READY, StatefulSet shows 0/1

IMPACT: Reduced HA capability, replica may not serve reads

Step-by-step diagnosis:

Step 1:

```
kubectl describe pod pgkubcluster-i2-d6hm-0 -n postgres-operator
```

Step 2:

```
kubectl get pod pgkubcluster-i2-d6hm-0 -n postgres-operator -o jsonpath='{.status.containerStatuses[*].name}
{.status.containerStatuses[*].ready}'
```

Step 3:

```
kubectl logs pgkubcluster-i2-d6hm-0 -c -n postgres-operator --tail=50
```

Step 4:

```
kubectl logs pgkubcluster-i2-d6hm-0 -c pmm-client -n postgres-operator --tail=50
```

Step 5:

```
kubectl logs pgkubcluster-i2-d6hm-0 -c pgbackrest -n postgres-operator --tail=50
```

Common causes and fixes:

- PMM client can't reach PMM server: Check pmm-0 pod is healthy and network policies allow access
- pgBackRest config issue: Verify backup repository secret exists and is correctly referenced
- Resource limits exceeded: Check if the container is OOMKilled (describe pod will show this)
- Certificate expiry: TLS cert for replication may need renewal
- Quick fix: Delete the pod to trigger recreation: `kubectl delete pod pgkubcluster-i2-d6hm-0 -n postgres-operator`

Issue 2: salt-control-postgres pgBouncer at 0/0 Replicas

Symptoms

OBSERVED: deployment salt-control-postgres-pgbouncer READY: 0/0, DESIRED: 0

IMPACT: No connection pooling, direct connections bypass pgBouncer

Diagnosis and fix:

Check CR: `kubectl get perconapgcluster salt-control-postgres -n postgres-operator -o yaml | grep -A5 pgBouncer`

Fix: Scale up: Edit CR to set `spec.proxy.pgBouncer.replicas: 3`

Apply: `kubectl apply -f cr.yaml -n postgres-operator`

Verify: `kubectl get deployment salt-control-postgres-pgbouncer -n postgres-operator`

Issue 3: Failed pgbackupmonitor Jobs (Cluster 1)

Symptoms

OBSERVED: 3 pgbackupmonitor jobs Failed (ages 19d, 18d, 17d), recent ones OK

IMPACT: Monitoring gap during failure period, backups themselves likely unaffected

Diagnosis:

```
kubectl describe job pgkubcluster-pgbackupmonitor-29518550 -n postgres-operator
```

```
kubectl logs job/pgkubcluster-pgbackupmonitor-29518550 -n postgres-operator
```

Since recent jobs are succeeding, this was likely a transient issue (network, resource pressure, or node eviction). Clean up old failed jobs:

```
kubectl delete job -n postgres-operator -l pg-cluster=pgkubcluster --field-selector status.successful=0
```

10 Quick Reference & Commands

Essential Commands for Your Clusters

Action	Cluster 1 (pgkubcluster)	Cluster 2 (salt-control-postgres)
Check cluster status	kubectl get perconapgcluster pgkubcluster -n postgres-operator	kubectl get perconapgcluster salt-control-postgres -n postgres-operator
List all pods	kubectl get pods -n postgres-operator -l pg-cluster=pgkubcluster	kubectl get pods -n postgres-operator -l pg-cluster=salt-control-postgres
Patroni status	kubectl exec pgkubcluster-i1-rvcz-0 -c database -- patronictl list	kubectl exec salt-control-postgres- i1-hpkf-0 -c database -- patronictl list
Interactive psql	kubectl exec -it pgkubcluster- i1-rvcz-0 -c database -- psql	kubectl exec -it salt-control-postgres- i1-hpkf-0 -c database -- psql
View operator logs	kubectl logs deploy/percona- postgresql-operator -n postgres-operator	Same (shared operator)
Trigger backup	kubectl apply -f backup.yaml -n postgres-operator	Same pattern, change pgCluster name
Check backups	kubectl get perconapgbbackup -n postgres-operator	Same command (shows all)
pgBackRest info	kubectl exec pgkubcluster- repo-host-0 -c pgbackrest -- pgbackrest info	kubectl exec salt-control-postgres- repo-host-0 -c pgbackrest -- pgbackrest info

Recommended Patroni Parameters for Production CR

Parameter	Recommended Value	Purpose
deadlock_timeout	1s	Time before checking for deadlocks
log_lock_waits	on	Log queries waiting for locks > deadlock_timeout
lock_timeout	30s	Abort queries waiting for lock > 30s
statement_timeout	300000	Kill queries running > 5 minutes (ms)
idle_in_transaction_session_timeout	60000	Kill idle-in-transaction sessions > 1 min
log_min_duration_statement	1000	Log queries slower than 1 second
shared_buffers	25% of RAM	Main memory cache for data
effective_cache_size	75% of RAM	Planner hint for OS cache

work_mem	64MB	Memory for sorts/hashtables per operation
maintenance_work_mem	512MB	Memory for VACUUM/CREATE INDEX
max_connections	200	Max client connections (via pgBouncer)
wal_level	replica	Required for streaming replication (default)
huge_pages	try	Use huge pages if available (v2.8.0+)

Resources & Links

Documentation: <https://docs.percona.com/percona-operator-for-postgresql/>

GitHub: <https://github.com/percona/percona-postgresql-operator>

Community Forum: <https://forums.percona.com/>

Percona Everest (Web UI): <https://docs.percona.com/everest/>

JIRA Bug Reports: <https://perconadev.atlassian.net/>